

UNIVERSIDAD DON BOSCO

Facultad de Ingeniería — Escuela de Computación
Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente

PROYECTO DE CÁTEDRA

Campus Explorer UDB 2026

Mapa Interactivo del Campus Universitario

Docente	Ing. Aida Quintanilla
Ciclo	01 / 2026
Grupo	G01T
Fecha de entrega	27 de Mayo, 2026

INTEGRANTES DEL GRUPO

Anaya Brizuela, Bryan Ernesto	AB250136
Cabezas Alfaro, Ever Gabriel	CA240297
Cortez Duran, Diego Alejandro	CD231127
Miranda Cuellar, Concepción Getsemaní	MC250288
Urías Viscarra, Dania Merari	UV253195

TABLA DE CRITERIOS DE EVALUACIÓN

Criterio	%	Estado
Presentación (portada, redacción, ortografía)	10%	Cumplido
Mapa Interactivo (100%)	30%	Cumplido
Correcciones (si las hubiera)	10%	Cumplido
Expresiones Regulares	10%	Cumplido
jQuery	10%	Cumplido
AJAX	10%	Cumplido
localStorage / sessionStorage	10%	Cumplido
AngularJS	10%	Cumplido

1. MAPA INTERACTIVO (30%)

El mapa interactivo del campus UDB fue implementado mediante un SVG vectorial personalizado incrustado en `mapa.html`, cada edificio es un elemento `<path>` con id único. Al pasar el cursor sobre un edificio (`mouseover`) se despliega un panel lateral con imagen y descripción; al hacer clic se redirige a la página de detalle del edificio. En dispositivos móviles, el evento se adapta a click táctil.

Vista del Mapa del Campus

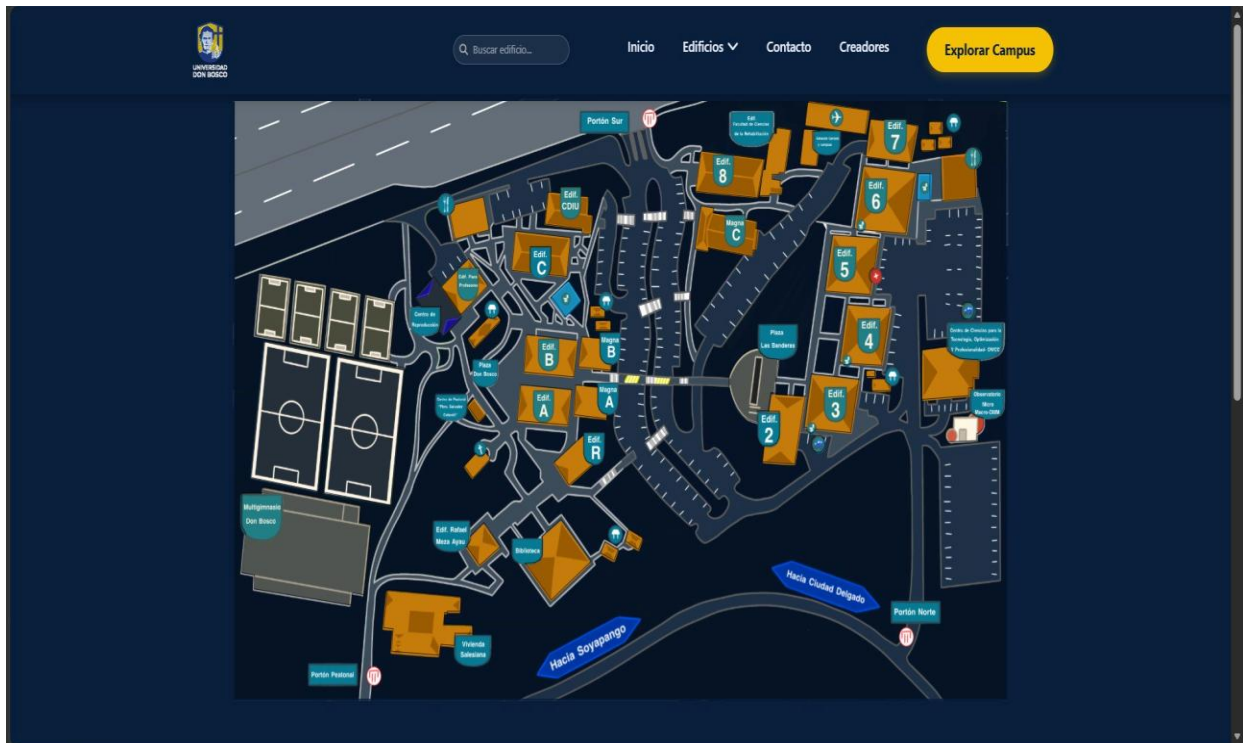


Figura 1.1 — Mapa interactivo del campus UDB con todos los edificios etiquetados

Estructura HTML del mapa — `mapa.html` (L28-L80)

El contenedor `.mapa-container` aloja el SVG con `viewBox="0 0 1965 1163"`. Cada edificio se representa como un `<path>` con su id (`Edif_A`, `Edif_B`, `Biblioteca`, `Edif_CDIU`, etc.), lo que permite la selección precisa mediante jQuery y la asignación de eventos interactivos por JavaScript.

```

html > mapa.html > ...
2 <html lang="es">
21 <body>
28 <main class="main-content">
29 <div class="mapa-container">
30 <svg viewBox="0 0 1965 1163" xmlns="http://www.w3.org/2000/svg" class="mapa-svg">
31 <g id="Capa_Edificios">
32 <!-- ... todos tus paths anteriores se mantienen igual ... -->
33 <path id="Edif_A" d="M736 527L743 454L873.5 465.5L863 536.5L736 527Z" stroke="black" />
34 <path id="Vivienda_Salesiana"
35 d="M977 1024.5L387 950L575 963L573 992.5H539.5V995.5C537.167 995.333 532.6 995.1 533 995.5C533.4 995.9 531.833 1010 531 1017L604 1018.5 605
36 stroke="black" />
37 <path id="Biblioteca"
38 d="M830.5 814L781.5 847L783.5 848.5V883L777.5 892L762 898L739 899L805.5 970.5L906 896.5V893.5L830.5 814Z"
39 stroke="black" />
40 <path id="Edif_Rafael_Meza_Ayau"
41 d="M633.5 820L599.5 846V853.5L596 858.5L589 866.5L622.5 903L675.5 866.5L633.5 820Z"
42 stroke="black" />
43 <path id="Igles"
44 d="M505 708L613 685L617 688L621 690L627 690.5L632.5 688.5L639.5 683L648.5 693V697L607 732H604L585 711.5V708Z"
45 stroke="black" />
46 <path id="Edif_B" d="M813 702.5L888 649V685L891 694L897 701L912 706L850.5 747L813 702.5Z"
47 stroke="black" />
48 <path id="Centro_de_Pastoral" d="M592 593L609 577L640 608L620 623L592 593Z" stroke="black" />
49 <path id="Edif_B" d="M719 621.5L728 551.5L858 561L847.5 630L719 621.5Z" stroke="black" />
50 <path id="Magna_A"
51 d="M864.5 610L868.5 552L924 555.5V587.5L925 592.5L929 600L936.5 606L948 609L946.5 618L864.5 610Z"
52 stroke="black" />
53 <path id="Mesa_Plaza"
54 d="M610 479L594.5 461L646 420L649.5 422L653 422.5H659.5L669 416.5L677 425.5L610 479Z"
55 stroke="black" />
56 <path id="Mesa_Biblio_Uno" d="M932 879L943 857L977 872.5L965.5 894L932 879Z" stroke="black" />
57 <path id="Mesa_Biblio_Dos" d="M995 856L1004 832.5L1039.5 849.5L1028.5 872.5L995 856Z"
58 stroke="black" />
59 <path id="Banos_Edif_C" d="M806.5 381L849 349.5L882.5 388L842 419.5L806.5 381Z" stroke="black" />
60 <path id="Magna_B"
61 d="M804 458.5L929.5 461L930.5 495.5L934.5 503L946 511L959 512L957 523.5L875 517L884 458.5Z"
62 stroke="black" />
63 <path id="Mesa_Mag_B_Uno"
64 d="M905.5 419.5V398.5H930.5L931 399.5L932.5 402L936.5 404.5L942.5 406.5V419.5H905.5Z"
65 stroke="black" />
66 <path id="Mesa_Mag_B_Dos" d="M918.5 445.5V426.5H957.5V445.5H918.5Z" stroke="black" />
67 <path id="Edif_Profesores"
68 d="M575 415L525.5 361L567 328V323.5H572L595 306.5L612 324H621L621.5 334L643.5 359L575 415Z"
69 stroke="black" />
70 <path id="Centro_de_Regpro"
71 d="M505 341L458 382.5L473.5 400H523V436L518 447.5L530 458.5L577 420L505 341Z" stroke="black" />
72 <path id="Comedor_Profe"
73 d="M559.5 268.5L547 220.5L556.5 212.5L559.5 201.5L632 186.5L647 250.5L559.5 268.5Z"
74 stroke="black" />
75 <path id="Edif_C"
76 d="M704.5 329L714 253L854.5 265.5L845 341L799.5 337.5L801.5 333L804.5 327L805 283H751.5V327L753.5 333L704.5 329Z"
77 stroke="black" />
78 <path id="Edif_CDII"
79 d="M700 239.5L798 177L829 178.5V166.5H802V185.5H899.5V221.5H909.5L907.5 258H889L884.5 255.5L883.5 248L790 239.5Z"
80 stroke="black" />

```

Figura 1.2 — mapa.html: estructura del SVG con paths de edificios (L28-L80)

Datos de edificios y lógica de eventos — mapa.js (L27-L103)

El objeto datosEdificios en mapa.js contiene por cada edificio: nombre, array de imágenes, descripción y enlace. La función asignarEventosEdificios() usa \$('path').each() para registrar eventos en cada elemento SVG del mapa. Se detecta si el dispositivo es móvil para usar clic en lugar de mouseover.

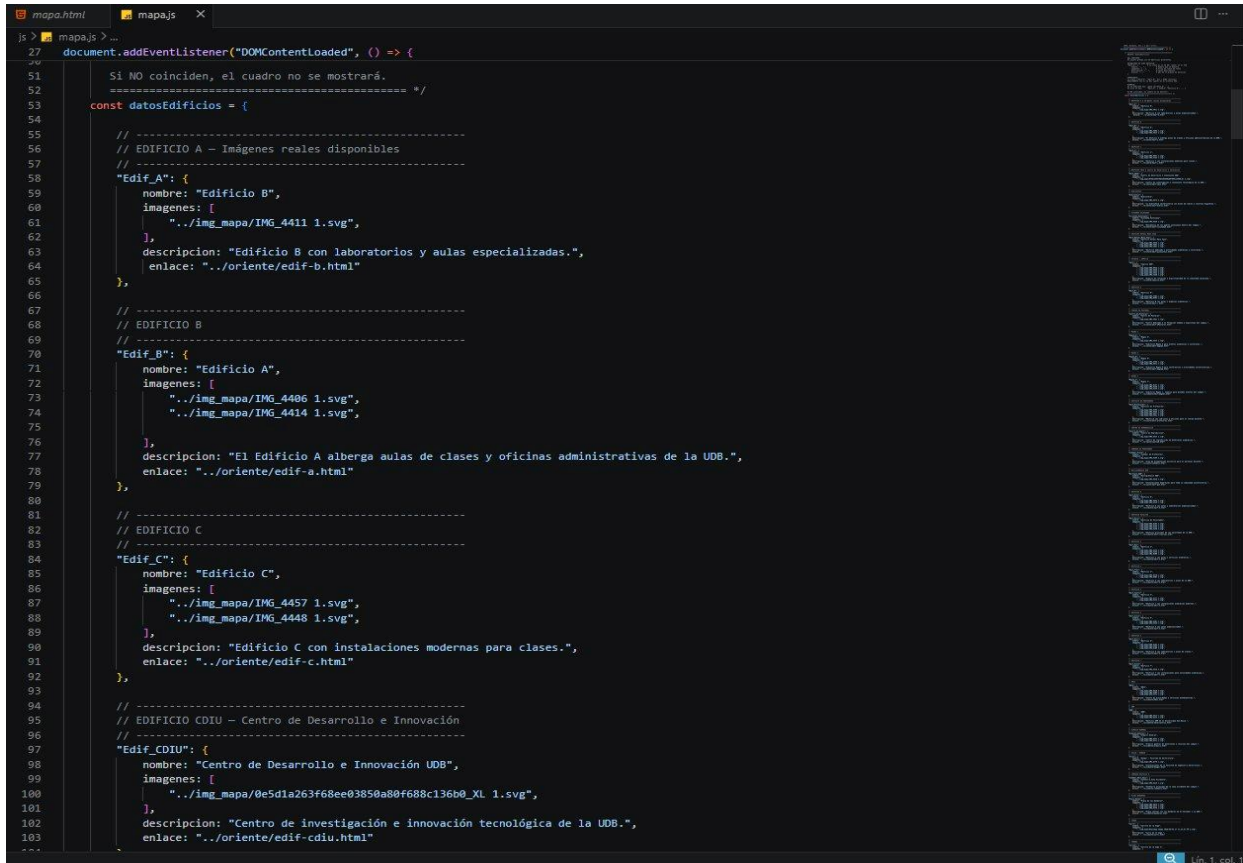


Figura 1.3 — mapa.js: objeto datosEdificios con información de edificios (L27-L103)

2. EXPRESIONES REGULARES (10%)

Se implementó un buscador de edificios en `buscador.js` que utiliza expresiones regulares para búsquedas tolerantes a acentos y mayúsculas/minúsculas. La función `buscarEdificios()` construye dinámicamente un `RegExp` normalizando el término de búsqueda: escapa caracteres especiales y reemplaza vocales con sus variantes acentuadas antes de crear el `regex`.

Implementación en `buscador.js` (L129-L144)

Procesos:

- (1) Se escapa el término con `.replace()` para caracteres especiales de `regex`.
- (2) Se normalizan vocales: `/a/gi` → `[aáÁ]`, `/e/gi` → `[eéÉ]`, etc.
- (3) Se construye `new RegExp(terminoLimpio, 'i')`.
- (4) Se filtra el catálogo con `regex.test(edificio.nombre)`. Esto garantiza encontrar resultados sin importar si el usuario escribe con o sin acentos.

```
const regex = new RegExp(terminoLimpio, 'i');
return catalogoEdificios.filter(edificio => regex.test(edificio.nombre));
```



```
js > buscador.js > ...
129 function buscarEdificios(termino) {
130   if (!termino.trim()) return [];
131
132   // Construye una expresión regular que ignore
133   // mayúsculas/minúsculas y acentos básicos
134   const terminoLimpio = termino
135     .replace(/[.*+?^${}()|[\]\\]/g, '\\$&') // escapa caracteres especiales
136     .replace(/a/gi, '[aáÁ]')
137     .replace(/e/gi, '[eéÉ]')
138     .replace(/i/gi, '[iíÍ]')
139     .replace(/o/gi, '[oóÓ]')
140     .replace(/u/gi, '[uúÚ]');
141
142   const regex = new RegExp(terminoLimpio, 'i');
143   return catalogoEdificios.filter(edificio => regex.test(edificio.nombre));
144 }
145
```

Figura 2.1 — `buscador.js`: construcción dinámica de `RegExp` con normalización de acentos (L129-L144)

3. jQuery — Selectores, Filtros y Eventos (10%)

jQuery se utiliza para:

- (a) selección y manipulación de elementos SVG del mapa mediante \$('path'),
- (b) registro de eventos clic y mouseover en los edificios,
- (c) manejo del panel de información, y
- (d) carga asíncrona de datos mediante \$.getJSON(). Su uso es no invasivo: toda la lógica está en archivos .js externos, separada del HTML.

Eventos en edificios del mapa — mapa.js

La función asignarEventosEdificios() usa \$('path').each() y .off('click mouseover') para limpiar y reasignar eventos. \$edificio.on('mouseover') activa el panel en escritorio; \$edificio.on('click') lo hace en móvil. Esto muestra el manejo adecuado de selectores y filtros jQuery para eventos dinámicos.

```
// =====
// FUNCIÓN: asignarEventosEdificios
// =====
function asignarEventosEdificios(datosEdificios) {
  // Elimina eventos anteriores y reasigna
  $('path').off('click mouseover');

  $('path').each(function () {
    const $edificio = $(this);

    if (esMobile()) {
      $edificio.on('click', function (e) {
        e.stopPropagation();
        activarEdificio($edificio, datosEdificios);
      });
    } else {
      $edificio.on('mouseover', function () {
        activarEdificio($edificio, datosEdificios);
      });
      $edificio.on('click', function (e) {
        e.stopPropagation();
        activarEdificio($edificio, datosEdificios);
      });
    }
  });
}

// =====
// RESIZE: reasigna eventos al cambiar tamaño
// =====
$(window).on('resize', function () {
  cerrarCuadro();
  // datosEdificios ya están en el closure del $.getJSON
  // Se reasignan mediante la variable guardada abajo
  if (window._datosEdificiosUDB) {
    asignarEventosEdificios(window._datosEdificiosUDB);
  }
});
```

Figura 3.1 — mapa.js: jQuery para eventos click/mouseover en paths SVG del campus

AJAX con jQuery — \$.getJSON() (mapa.js — L827)

\$.getJSON('./js/edificios.json') realiza la petición GET asíncrona. .done() recibe los datos parseados y los asigna a window._datosEdificiosUDB para reutilización en resize. .fail() captura errores con console.error(). Este patrón demuestra el uso correcto de AJAX y promesas jQuery.


```
/* =====  
AJAX: Carga los datos desde edificios.json  
usando $.getJSON (jQuery + AJAX)  
  
FLUJO:  
1. Hace petición GET a ../js/edificios.json  
2. jQuery parsea el JSON automáticamente  
3. Se llama a asignarEventosEdificios con los datos  
4. Si falla, muestra error en consola  
===== */  
$.getJSON('../js/edificios.json')  
  .done(function (datosEdificios) {  
    // Guarda en variable global para poder reasignar en resize  
    window._datosEdificiosUDB = datosEdificios;  
    // Inicializa los eventos con los datos cargados  
    asignarEventosEdificios(datosEdificios);  
  })  
  .fail(function (jqXHR, textStatus, errorThrown) {  
    console.error('Error al cargar edificios.json:', textStatus, errorThrown);  
  });  
  
// Fin $(document).ready
```

Figura 3.2 — *mapa.js*: *\$.getJSON()* con cadena *.done()/fail()* (L827)

4. AJAX (10%)

La petición AJAX se realiza mediante \$.getJSON() de jQuery al archivo edificios.json. Esta llamada asíncrona permite que el mapa se inicialice y sea funcional sin bloquear el hilo principal del navegador. Los datos del JSON se procesan en el callback .done() y se distribuyen al sistema de eventos del mapa.

Flujo completo de la petición AJAX

1. Al dispararse DOMContentLoaded, se ejecuta \$.getJSON('../js/edificios.json').
2. jQuery realiza la petición GET y parsea automáticamente la respuesta JSON.
3. .done(): los datos se guardan en window._datosEdificiosUDB y se llama a asignarEventosEdificios().
4. .fail(): si hay error, se muestra en consola con textStatus y errorThrown.
5. Al redimensionar la ventana, \$(window).on('resize') reasigna eventos usando los datos guardados.

```

/* =====
AJAX: Carga los datos desde edificios.json
usando $.getJSON (jQuery + AJAX)

FLUJO:
1. Hace petición GET a ../js/edificios.json
2. jQuery parsea el JSON automáticamente
3. Se llama a asignarEventosEdificios con los datos
4. Si falla, muestra error en consola
===== */
$.getJSON('../js/edificios.json')
  .done(function (datosEdificios) {
    // Guarda en variable global para poder reasignar en resize
    window._datosEdificiosUDB = datosEdificios;
    // Inicializa los eventos con los datos cargados
    asignarEventosEdificios(datosEdificios);
  })
  .fail(function (jqXHR, textStatus, errorThrown) {
    console.error('Error al cargar edificios.json:', textStatus, errorThrown);
  });

// Fin $(document).ready

```

Figura 4.1 — Implementación AJAX: \$.getJSON() con manejo de promesas .done()/fail()

5. localStorage / sessionStorage (10%)

El formulario de contacto implementa persistencia completa con localStorage. Los mensajes enviados se serializan como JSON y se almacenan bajo la clave 'mensajes_contacto_udb'. La implementación permite crear, leer, actualizar y eliminar mensajes (CRUD completo). Tanto contacto.js como angular-contacto.js acceden a este almacenamiento de forma coordinada.

Funciones de lectura y escritura — contacto.js (L72-L93)

KEY_STORAGE = 'mensajes_contacto_udb' centraliza la clave. cargarMensajesDeStorage() usa localStorage.getItem(KEY_STORAGE) y JSON.parse() para recuperar datos. guardarMensajesEnStorage() usa localStorage.setItem(KEY_STORAGE, JSON.stringify(mensajesGuardados)) para persistir. Ambas funciones encapsulan correctamente la lógica de almacenamiento.

```
localStorage.setItem(KEY_STORAGE, JSON.stringify(mensajesGuardados));
const datos = localStorage.getItem(KEY_STORAGE);
```

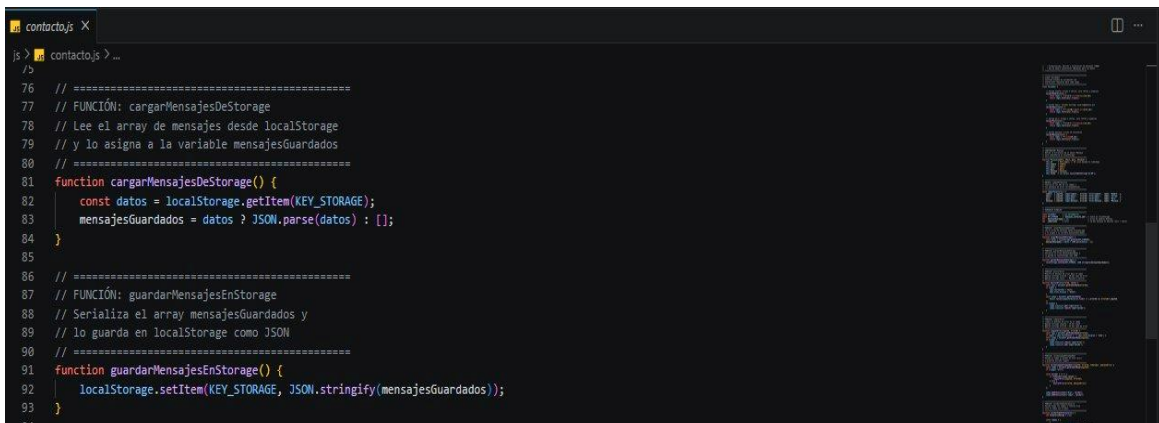


Figura 5.1 — contacto.js: implementación de localStorage para CRUD de mensajes (L76-L93)

Integración con AngularJS — angular-contacto.js (L19)

El controlador MensajesCtrl también lee desde localStorage mediante localStorage.getItem(KEY_STORAGE) en la función cargarMensajes(). Esto sincroniza los datos del storage con \$scope.mensajes, permitiendo que la tabla de mensajes en la vista Angular se actualice reactivamente cuando hay cambios.



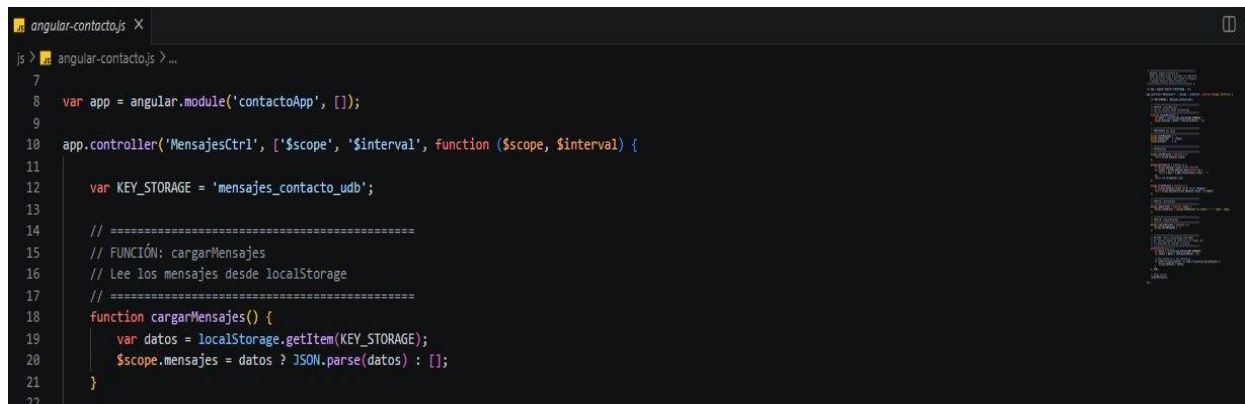
Figura 5.2 — angular-contacto.js: lectura de localStorage sincronizada con \$scope (L19)

6. AngularJS (10%)

AngularJS se implementó en la sección de contacto. El módulo contactoApp con su controlador MensajesCtrl gestiona: visualización reactiva de mensajes guardados, estadísticas de contacto (total, pendientes, resueltos), filtrado por texto, ordenamiento de tabla, y sincronización con localStorage. Se usan las directivas ng-controller, ng-repeat, ng-bind, ng-model y \$interval.

Módulo y Controlador — angular-contacto.js (L8-L20)

angular.module('contactoApp', []) define el módulo raíz. El controlador MensajesCtrl se registra con inyección de dependencias ['\$scope', '\$interval']. La constante KEY_STORAGE en L12 centraliza la clave de acceso al localStorage. La función cargarMensajes() en L18 sincroniza los datos del storage con \$scope.mensajes.



```

angular-contacto.js X
js > angular-contacto.js > ...
7
8 var app = angular.module('contactoApp', []);
9
10 app.controller('MensajesCtrl', ['$scope', '$interval', function ($scope, $interval) {
11
12     var KEY_STORAGE = 'mensajes_contacto_udb';
13
14     // =====
15     // FUNCIÓN: cargarMensajes
16     // Lee los mensajes desde localStorage
17     // =====
18     function cargarMensajes() {
19         var datos = localStorage.getItem(KEY_STORAGE);
20         $scope.mensajes = datos ? JSON.parse(datos) : [];
21     }
22

```

Figura 6.1 — angular-contacto.js: módulo contactoApp y controlador MensajesCtrl (L8-L20)

Directivas AngularJS implementadas

Directiva	Ubicación	Propósito
ng-controller	contacto.html — L355	Vincula MensajesCtrl a la sección de mensajes
ng-bind	contacto.html — L368	Muestra estadísticas reactivas (total, pendientes)
ng-repeat	contacto.html — L431	Renderiza la tabla de mensajes guardados
ng-model	contacto.html	Binding bidireccional de campos del formulario
\$scope.mensajes	angular-contacto.js	Arreglo reactivo sincronizado con localStorage
\$interval	angular-contacto.js	Actualización periódica automática de mensajes

7. CONTENIDO ADICIONAL — El Legendario Carro de la Inge

Como sección adicional creativa, se incorporó una página dedicada al legendario automóvil de la Ing. Aida Quintanilla: 'El Carrito de la Inge'. Sigue la misma estructura de detalle de edificios (hero + contenido + galería + tabla de información), y está referenciada desde el mapa SVG como un punto de interés del campus, demostrando la flexibilidad del sistema implementado.

Vista de la página del Carro



Figura 7.1 — Vista de la página dedicada al Carro de la Inge en el sitio web

Estructura HTML — carro.html

La página reutiliza exactamente la misma estructura de las páginas de edificios: sección .hero con imagen SVG, sección .contenido-historia con bloques de texto descriptivo, mini galería con video e imágenes, y tabla de información general (Propietaria, Categoría, Características). Esto evidencia la consistencia y reusabilidad del diseño del sistema.

```

1  <html lang="es">
2  <body>
26  <!-- ===== CONTENIDO PRINCIPAL ===== -->
27  <main class="main-content">
28    <section class="historia">
29      <div class="hero">
30        
31        <div class="hero-text">
32          <h1>El Legionario Carro de la Inge</h1>
33          <p>
34            Más que un simple vehículo, este icónico automóvil se ha convertido en una leyenda dentro del campus UDB.
35            Elegancia, presencia y estilo se combinan perfectamente en el carro de la ingeniera, demostrando que el buen
36            gusto también se refleja sobre ruedas. Reconocido por estudiantes y admirado por todos, este auto representa
37            dedicación, profesionalismo y una vibra inconfundible que ya forma parte de la experiencia universitaria.
38          </p>
39        </div>
40      </div>
41
42      <!-- contenido -->
43      <div class="contenido-historia">
44        <!-- Historia -->
45        <div class="block">
46          <h2>Una Leyenda del Campus</h2>
47          <p>
48            Hablar del campus sin mencionar el carro de la Inge sería imposible. Su presencia destaca inmediatamente
49            entre
50            todos los vehículos de la universidad gracias a su estilo impecable y su apariencia siempre elegante. Cada
51            vez
52            que aparece en el estacionamiento, automáticamente roba miradas y se convierte en tema de conversación.
53          </p>
54          <p>
55            La ingeniera no solo destaca por su excelencia académica y su dedicación a la enseñanza, sino también por el
56            increíble estilo que transmite dentro y fuera del aula. Su automóvil refleja perfectamente su personalidad:
57            moderno, confiable, elegante y simplemente inolvidable.
58          </p>
59        </div>
60
61        <!-- mini galeria -->
62        <div class="block">
63          <h2>Galeria Exclusiva</h2>
64          <div class="galeria">
65            <video src="../video/WhatsApp Video 2026-05-21 at 5.41.05 PM.mp4" controls</video>
66            
67          </div>
68        </div>
69
70        <!-- datos en tablas -->
71        <div class="block">
72          <h2>Información General</h2>
73          <table>
74            <tr>
75              <th>Propietaria</th>
76              <td>La legendaria Ingeniera de DAW</td>
77            </tr>
78          </table>

```

Figura 7.2 — carro.html: estructura HTML siguiendo el patrón de páginas de edificios

Datos del Carro en mapa.js (L460-L482)

Los registros 'Carro' y 'Carro2' están definidos en el objeto datosEdificios con imágenes SVG, descripción y enlace a carro.html. Esto demuestra que el sistema del mapa interactivo puede incorporar cualquier punto de interés del campus más allá de edificios convencionales, siendo extensible por diseño.

```

27  document.addEventListener("DOMContentLoaded", () => {
53    const datosEdificios = {
459
460      // -----
461      // CARRO
462      // -----
463      "Carro": {
464        nombre: "Carrito de la Inge",
465        imagenes: [
466          "../img_mapa/WhatsApp Image 2026-05-03 at 12.23.51 PM 1.svg",
467        ],
468        descripcion: "Carro de la Inge.",
469        enlace: "../occidente/carro.html"
470      },
471
472      // -----
473      // CARRO2
474      // -----
475      "Carro2": {
476        nombre: "Carrito de la Inge 2",
477        imagenes: [
478          "../img_mapa/WhatsApp Image 2026-05-03 at 12.23.51 PM (1) 1 (1).svg",
479        ],
480        descripcion: "Carro de la Inge.",
481        enlace: "../occidente/carro.html"
482      },
483    },

```

Figura 7.3 — mapa.js: entradas del Carro en datosEdificios (L460-L482)